

Movie Rental Database System

CS 157A — Final Project Report

Team Members:

Addison Schleigh — SID: 017595110

Haitham Assaf — SID: 017558515

Tyler Fabela — SID: 016347201

1. Introduction

This project is a Movie Rental Database System we built for CS 157A. The application lets store staff manage a movie catalog, register customers, process rentals and returns, and record customer ratings.

We built the system using a three-tier architecture: a Java Swing desktop GUI for the user (presentation layer), Java DAO classes that send SQL through JDBC (application layer), and a MySQL database normalized to BCNF (data layer).

Building it this way means each layer can change without breaking the others. For example, we could replace the Swing GUI with a web front-end and the database code would still work the same way.

2. Objectives

Primary Goals: Design a relational schema with at least four tables in BCNF, populate it with at least fifteen rows per table, and build a Java application that can run SELECT, INSERT, UPDATE, and DELETE on every table through JDBC.

Functionality Overview: Staff can browse the catalog, search for movies and customers, register new customers, rent movies, process returns with auto-calculated late fees, and record ratings.

3. Database Design

3.1 Entity-Relationship Model

The system has two strong entities (Customer and Movie), two associative entities (Rentals and Ratings), and one weak entity (Returns).

Customer and Movie are the main entities. A customer can rent many movies and a movie can be rented by many customers, so we made Rentals an associative entity that has its own attributes (rental date, due date, status). Ratings work the same way, many-to-many between Customer and Movie.

Returns is a weak entity because each return only exists in relation to a rental. Without a rental, a return would not have meaning.

Cardinalities:

Relationship	Connects	Cardinality
Makes	Customer → Rentals	1 : M

For	Rentals → Movie	N : 1
Has	Rentals → Returns	1 : 1
Gives	Customer → Ratings	1 : M
Of	Ratings → Movie	N : 1

3.2 Relational Schema

Each entity becomes a table. Relationship attributes plus the keys of the connected entities become foreign keys in the resulting tables.

Table	Attributes
Movies	MovieID (PK), Title, Genre, Director, ReleaseYear, MPAARating, CopiesAvailable, RentalPrice
Customers	CustomerID (PK), FirstName, LastName, Email (UNIQUE), Phone, Address, JoinDate
Rentals	RentalID (PK), CustomerID (FK), MovieID (FK), RentalDate, DueDate, Status
Returns	ReturnID (PK), RentalID (FK, UNIQUE), ReturnDate, LateFee
Ratings	RatingID (PK), CustomerID (FK), MovieID (FK), Score, ReviewDate

3.3 BCNF Justification

A table is in BCNF if every functional dependency $X \rightarrow Y$ has X as a superkey. We checked each table:

- Movies: only dependency is $\text{MovieID} \rightarrow (\text{rest})$. MovieID is the primary key. BCNF holds.
- Customers: CustomerID and Email are both unique, so both are superkeys. BCNF holds.
- Rentals: only dependency is $\text{RentalID} \rightarrow (\text{rest})$. BCNF holds.
- Returns: ReturnID and RentalID are both unique. Both are superkeys. BCNF holds.
- Ratings: only dependency is $\text{RatingID} \rightarrow (\text{rest})$. BCNF holds.

Since every dependency is on a superkey, the schema avoids the update, insert, and delete anomalies that motivate normalization.

4. Implementation

4.1 Three-Tier Architecture

The application has three independent layers:

- Presentation layer (ui package): A Swing GUI with a JFrame and JTabbedPane. Each of the five tabs corresponds to one table and shows a JTable with data plus buttons for Add, Edit, Delete, and Search.
- Application layer (dao and model packages): Plain Java model classes mirror table rows. DAO classes hold all the SQL. The GUI never builds SQL directly — it asks a DAO.

- Data layer (db package and the MySQL server): The DatabaseConnection class is the only place that opens a JDBC connection. The schema and sample data live in the movie_rental MySQL database.

4.2 JDBC Layer

Every database call goes through JDBC. We open a Connection from DriverManager, build a PreparedStatement with parameters, run it, and close everything in a try-with-resources block so connections always get released. A typical method looks like:

```
try (Connection conn = DatabaseConnection.getConnection();
    PreparedStatement pstmt = conn.prepareStatement(sql)) {
    pstmt.setInt(1, movieId);
    try (ResultSet rs = pstmt.executeQuery()) {
        while (rs.next()) movies.add(buildMovie(rs));
    }
}
```

We use PreparedStatement with parameters (?) instead of string concatenation. This protects against SQL injection and lets the database reuse the compiled query plan.

4.3 Required SQL Operations

Every DAO supports all four core SQL operations. Here is what they look like in MovieDAO:

Operation	Method	SQL
SELECT	getAllMovies()	SELECT * FROM Movies ORDER BY MovieID
SELECT	searchByTitle(kw)	SELECT * FROM Movies WHERE Title LIKE ?
INSERT	insertMovie(m)	INSERT INTO Movies (...) VALUES (?, ?, ...)
UPDATE	updateMovie(m)	UPDATE Movies SET ... WHERE MovieID = ?
DELETE	deleteMovie(id)	DELETE FROM Movies WHERE MovieID = ?

5. Functional Requirements

Users: Store staff are the main users. They access the system through the desktop GUI.

Features and how they work:

- Browse / Search Movies: Click the Movies tab to see the catalog. Type a keyword and click Search to find matching titles.
- Register a Customer: Click Add Customer on the Customers tab, fill in the form, click OK.

- Rent a Movie: Select a movie on the Movies tab, click Rent This Movie, pick a customer from a dropdown. The system inserts a rental and decrements CopiesAvailable.
- Process a Return: Select an active rental on the Rentals tab, click Process Return. The system auto-calculates a late fee (1.50 per day past due), inserts a Return record, marks the rental Returned, and increments CopiesAvailable.
- Submit a Rating: On the Ratings tab, click Add Rating, fill in CustomerID, MovieID, score (1-5), and date.
- Edit / Delete Records: Select a row, click Edit to open a pre-filled form. Delete asks for confirmation first.

6. Non-Functional Requirements

- Performance: Indexes are created automatically on primary keys and the UNIQUE Email column. Filters on Rentals and Ratings run on indexed columns.
- Security: Database credentials live in one class for easy rotation. All queries use PreparedStatement with parameter binding to prevent SQL injection.
- Maintainability: The three-tier separation means a change to the GUI does not affect the database code, and vice versa.
- Usability: Common actions are two clicks. Forms validate basic input before sending SQL. Destructive actions ask for confirmation.

7. Tools and Technologies

- Database: MySQL Server 9.7
- Database API: JDBC with the MySQL Connector/J driver
- Programming Language: Java (JDK 17 or later)
- UI Toolkit: Java Swing (JFrame, JTabbedPane, JTable, JOptionPane, JDialog)
- Development Environment: IntelliJ IDEA / Eclipse / VS Code
- Version Control: Git and GitHub

8. Conclusion

This project pulls together every major topic covered in CS 157A. We started with an entity-relationship diagram, mapped it to a relational schema, verified that every table is in BCNF, then built a Java application that talks to the database through JDBC inside a three-tier architecture.

The result is a working movie rental application that supports all four basic SQL operations on five normalized tables. Future extensions could include: replacing the Swing GUI with a web front-end (changing only the presentation layer), adding stored procedures for late-fee calculation, or adding role-based access for staff vs. managers. Each of these could be done without changing the schema, which is the point of building it this way.